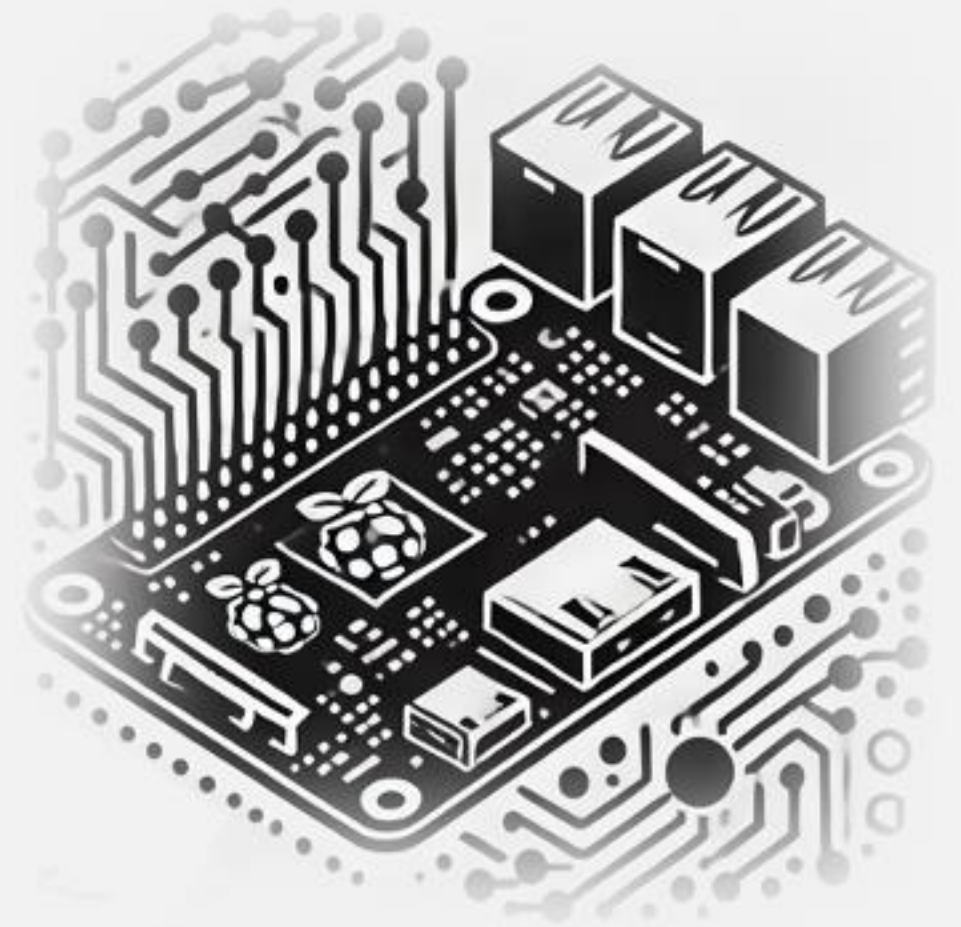


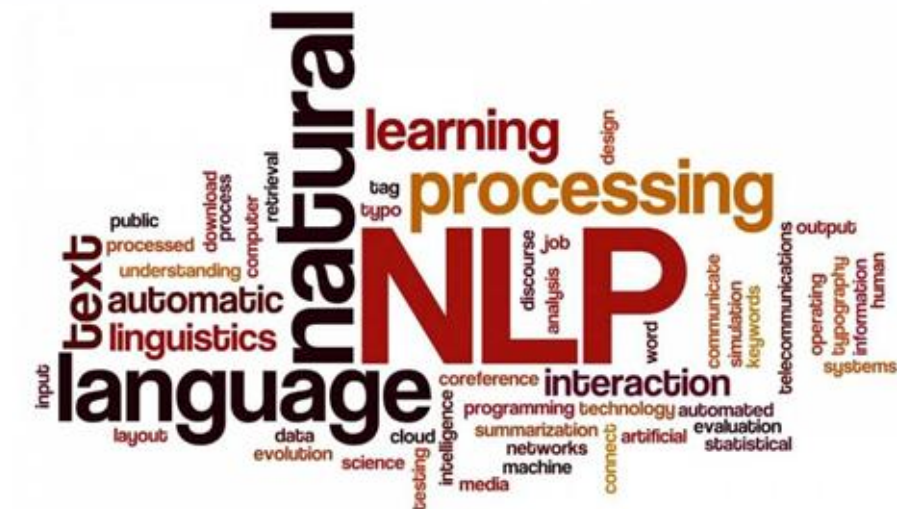
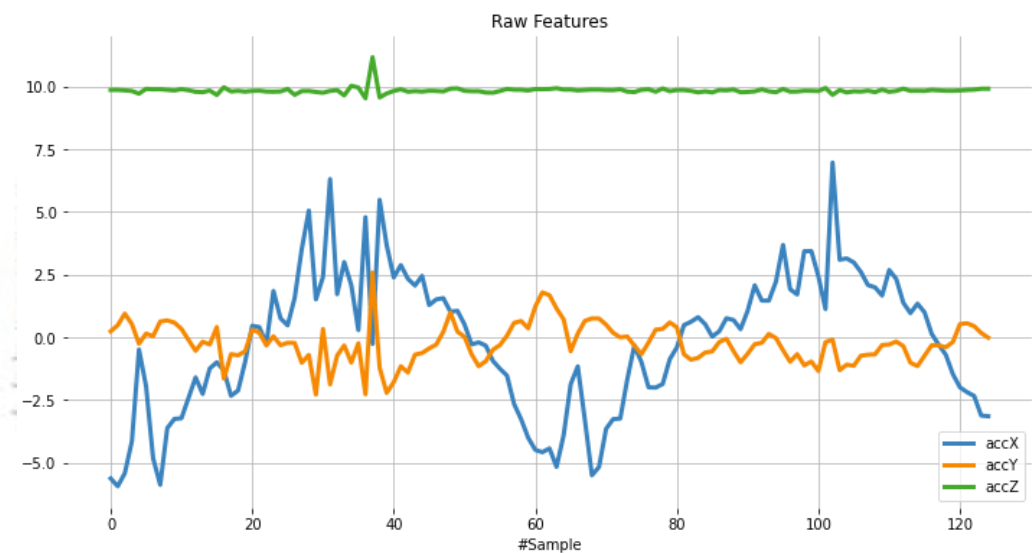
IESTI05 – Edge AI

Machine Learning System Engineering

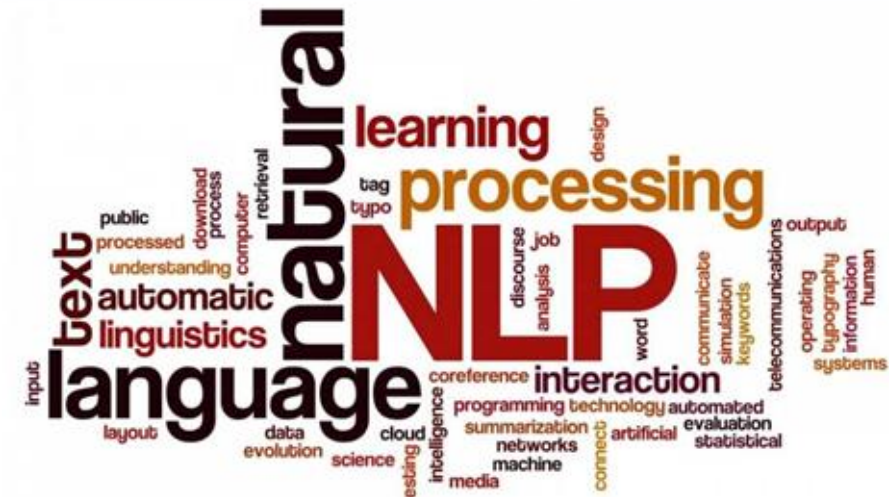
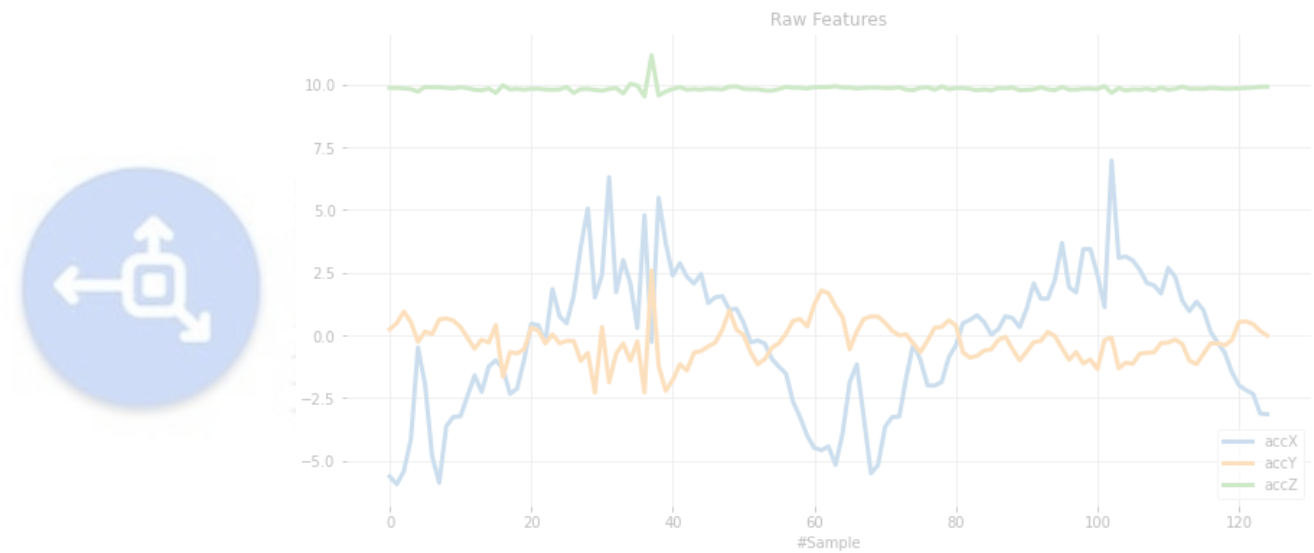
15. Text Generation – RNN The VerneBot



Unstructured Data



Unstructured Data - Text





Neural Network Architectures

Vibration
Analysis



Image
Classification



Text
Generation



Image
Generation



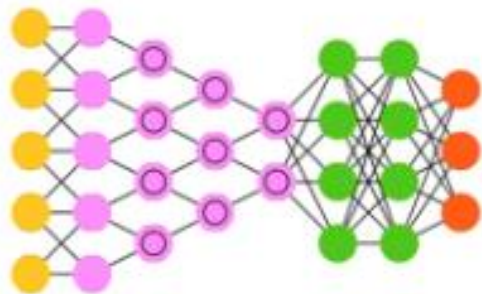
Large Language
Models- LLMs



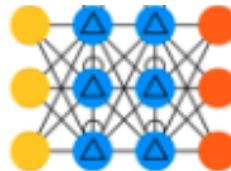
MLP - Deep Neural Network



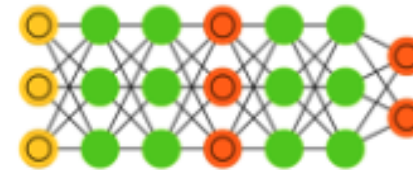
CNN - Convolutional NN



RNN - Recurrent NN (GRU/LSTM)



GAN - Generative Adversarial N.



AN - Attention (Transformers)





Neural Network Architectures

Vibration
Analysis



Image
Classification



Text
Generation



Image
Generation



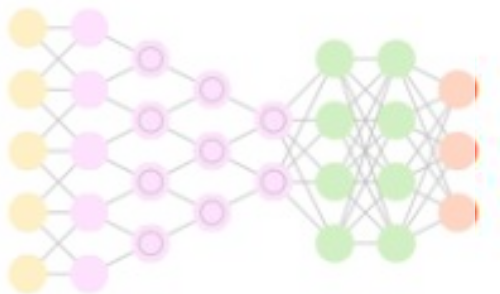
Large Language
Models- LLMs



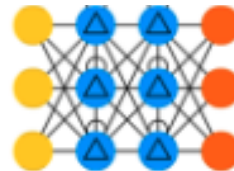
MLP - Deep Neural Network



CNN - Convolutional NN



RNN - Recurrent NN (GRU/LSTM)

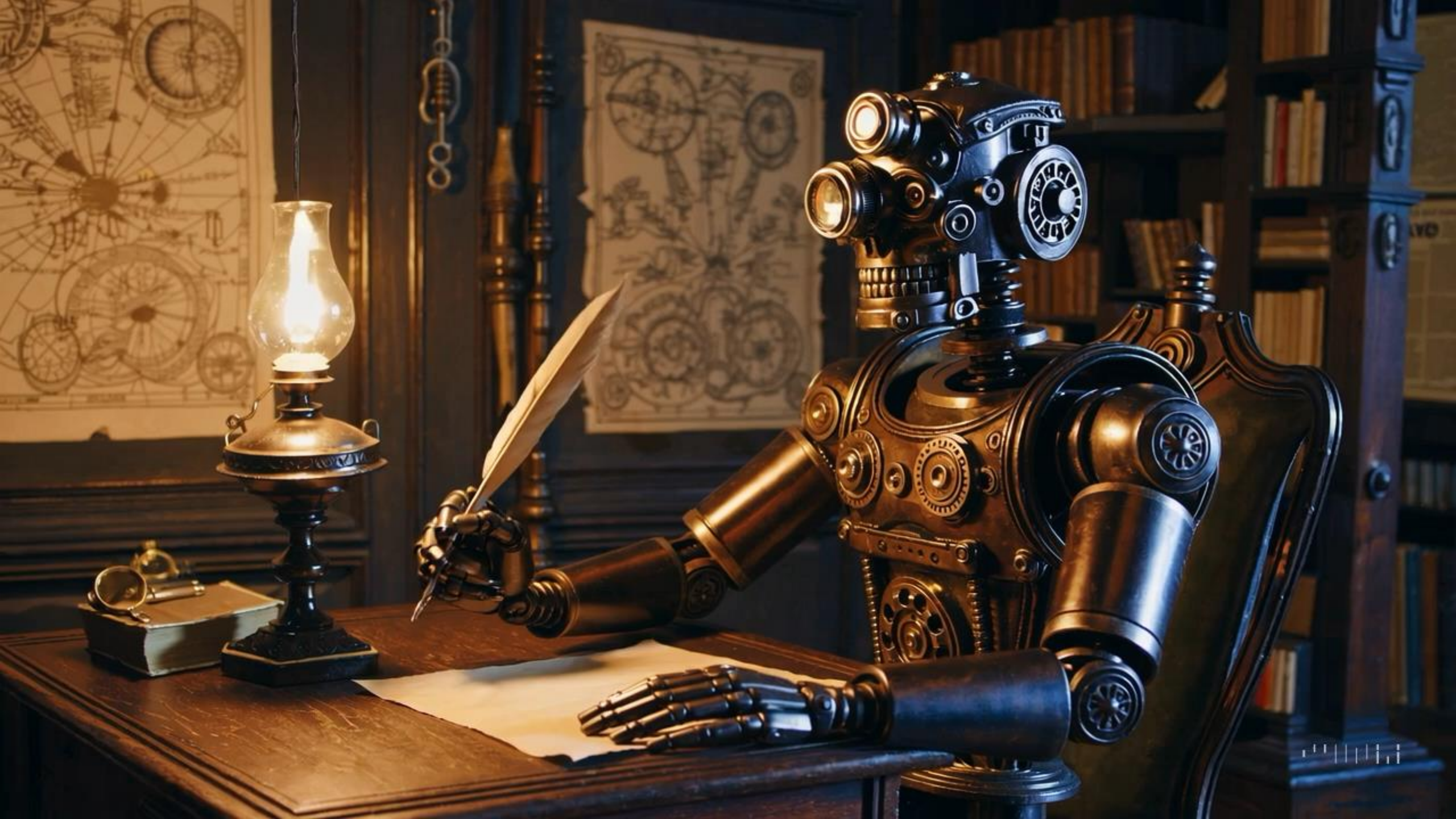


GAN - Generative Adversarial N.



AN - Attention (Transformers)





20_JulesVerneBot_Generatin x +

← → ↺ 🏠

https://colab.research.google.com/github/Mjrovai/TinyML4D/blob/main/WALC_2024/notebooks/20_JulesVerneBot_Generating_English_Texts_with...

🔍 ☆ 🗂 👤 ⋮

🔍 Commands

+ Code + Text

Connect T4 ⌵

🔗 ⚙️ ✨ ⌵

☰

🔍

<>

{x}

🔑

📁

🖼

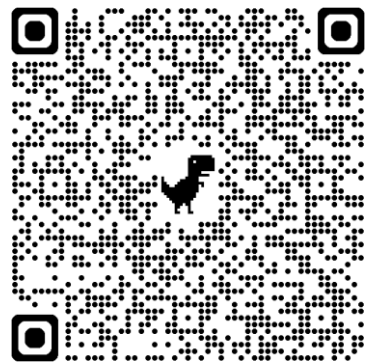
⌵

Generating Text with RNNs: The Jules Verne Bot

- By Marcelo Rovai @Nov24
- With code support from **Claude 3.5 Sonnet** and **ChatGPT 4o**



● ✕





Project Gutenberg

DATASET

Total characters:

- 5,768,791

Unique characters:

- 123



1. 'A Journey to the Centre of the Earth'
2. 'In Search of the Castaways'
3. 'An Antarctic Mystery'
4. 'In the year 2889'
5. 'Around the World in Eighty Days'
6. 'Michael Strogoff'
7. 'Five Weeks in a Balloon'
8. 'The Mysterious Island'
9. 'From the Earth to the Moon'
10. 'Twenty Thousand Leagues under the Sea'

TOKENIZATION AND VOCABULARY

Conversion of text
into numeric tokens.



Character-level
tokenization: 123
unique characters.



Example: "The Project" →
[122 52 69 66 1 48 79 76 71 66 64 81].

TOKENIZATION

```
['\\n', ' ', '!', '"', '#', '$', '%', '&', "'", '(', ')', '*', '+',  
, '-', '.', '/', '0', '1', '2', '3', '4', '5', '6', '7', '8',  
'9', ':', ';', '<', '=', '>', '?', 'A', 'B', 'C', 'D', 'E', 'F',  
'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S',  
'T', 'U', 'V', 'W', 'X', 'Y', 'Z', '[', ']', '_', 'a', 'b', 'c',  
'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p',  
'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', '{', '}', '~',  
'f', 'o', 'l', 'A', 'E', 'E', 'a', 'a', 'e', 'c', 'e', 'e',  
'e', 'i', 'n', 'o', 'u', 'u', 'e', 'e', 'o', 'o', 'o',  
'', '.', ',', '~', '™', '\\ufe0f'], dtype='<U1')
```

```
[31, 42, 40, 1, 30, 28, 46, 40, 48, 45, 45, 42, 0, 0, 36, 0, 0,  
31, 71, 1, 76, 65, 76, 77, 68, 71, 13, 0, 0, 48, 69, 57, 1, 70,  
71, 65, 76, 61, 1, 60, 61, 75, 76, 57, 75, 11, 1, 78, 65, 70, 60,  
71, 1, 60, 57, 1, 59, 65, 60, 57, 60, 61, 1, 72, 57, 74, 57, 1,  
71, 1, 32, 70, 63, 61, 70, 64, 71, 1, 41, 71, 78, 71, 11, 1, 61,  
70, 59, 71, 70, 76, 74, 61, 65, 1, 70, 71, 0, 76, 74, 61, 69, 1,  
60, 57, 1, 30, 61, 70, 76, 74, 57, 68, 1, 77, 69, 1, 74, 57, 72,  
57, 82, 1, 57, 73, 77, 65, 1, 60, 71, 1, 58, 57, 65, 74, 74, 71,  
11, 1, 73, 77, 61, 1, 61, 77, 1, 59, 71, 70, 64, 61])
```

TRAINING SEQUENCES



Goal: Predict the next character in a sequence.



Length of the sequence: 120 characters (~ paragraph).



Input 'Hello my nam'

Output 'ello my name'.

EMBEDDING

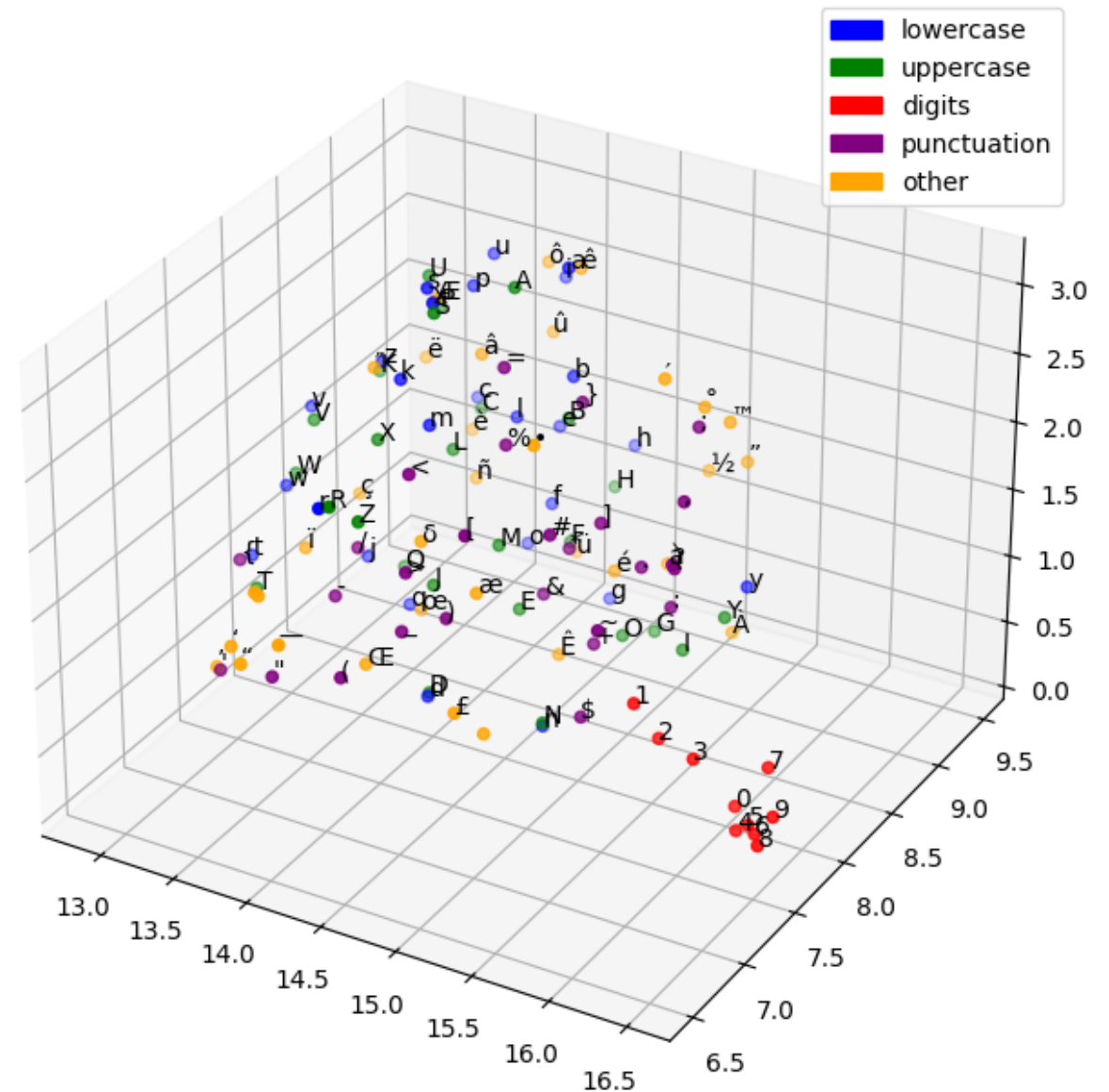
Each character is represented as a vector of 256 dimensions.



Embedding captures relationships between characters in dense vectors.

EMBEDDING

3D Projection of Character Embeddings Using UMAP



MODEL ARCHITECTURE

Embedding Layer:
Converts characters into
dense vectors.

A light blue downward-pointing arrow indicating the flow from the Embedding Layer to the RNN/GRU Layer.

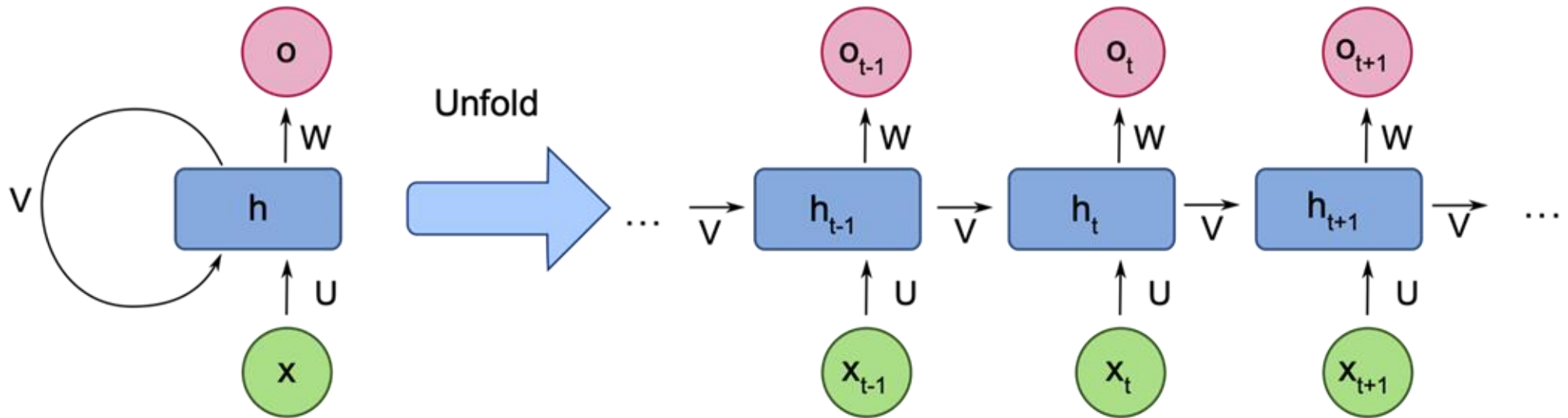
RNN/GRU Layer (1024
units): Learn from
sequences.

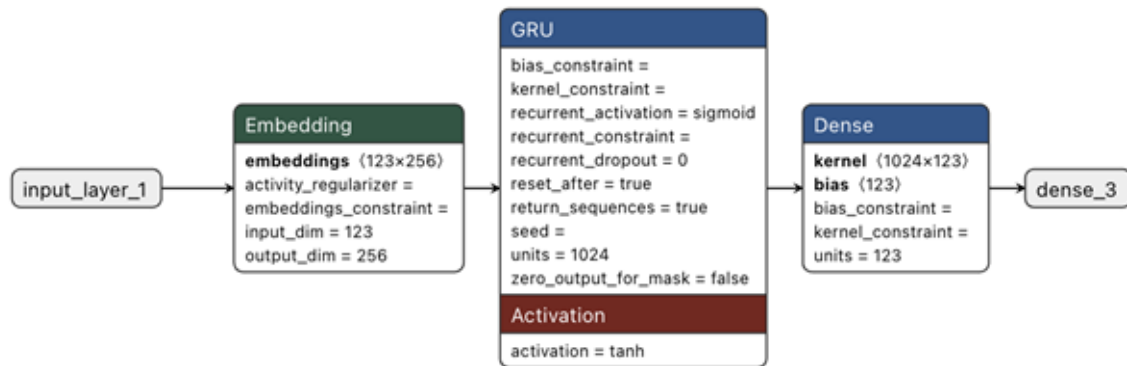
A light blue downward-pointing arrow indicating the flow from the RNN/GRU Layer to the Dense layer.

Dense layer: Generates
probabilities for each
character (123).

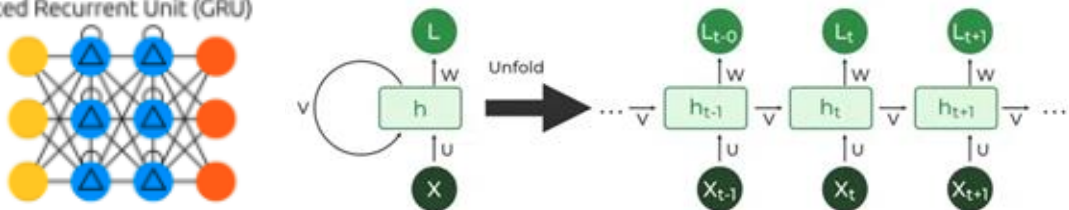
Deep Learning models (or artificial neural networks)

Recurrent Neural Networks (RNNs): Designed for **sequential data like time series or text**, these networks use their internal state (memory) to process sequences of inputs.





Gated Recurrent Unit (GRU)



Model: "sequential_4"

Layer (type)	Output Shape	Param #
embedding_4 (Embedding)	(1, 120, 256)	31,488
gru_3 (GRU)	(1, 120, 1024)	3,938,304
dense_3 (Dense)	(1, 120, 123)	126,075

Total params: 4,095,867 (15.62 MB)
 Trainable params: 4,095,867 (15.62 MB)
 Non-trainable params: 0 (0.00 B)

RNN MODEL (RECURRENT)

MODEL TRAINING

Loss Function: Categorical
Sparse Crossentropy

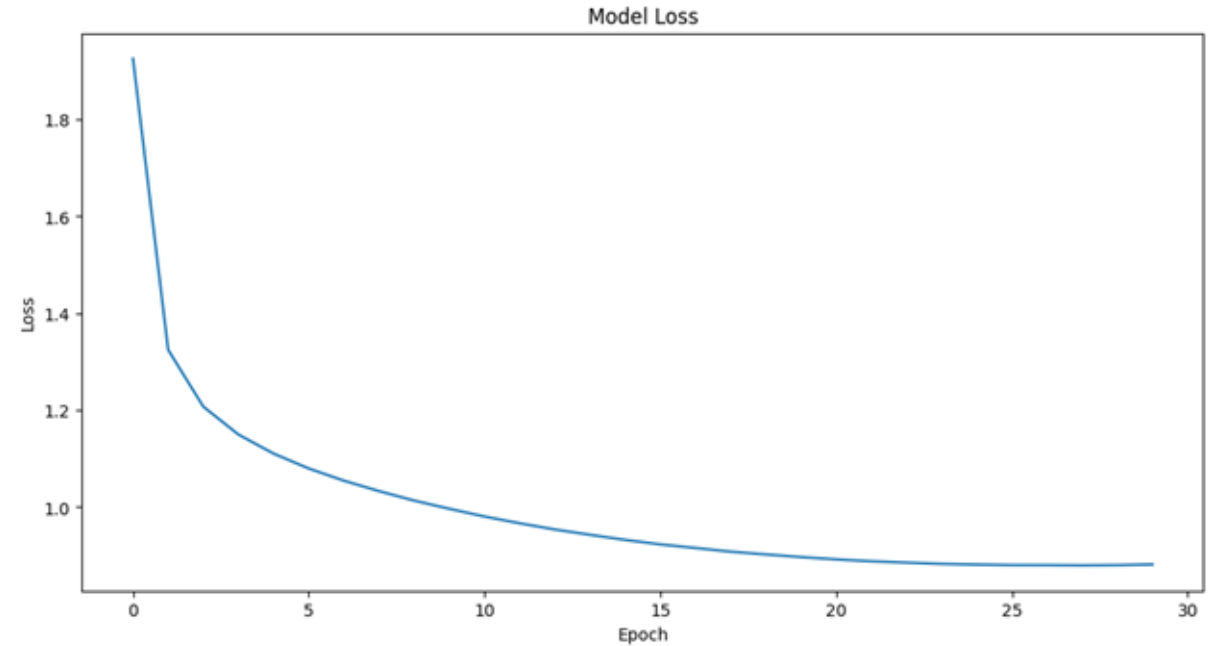
Optimizer: Adam

Epochs: 30

batch size: 128

buffer size: 10,000 (for
dataset shuffling)

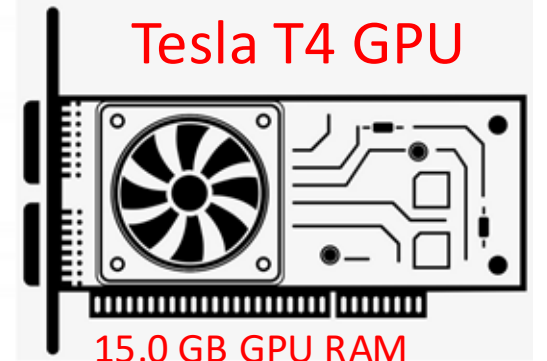
Monitoring loss reduction
over time.



(33 minutes for training)

Google
colab

Tesla T4 GPU



15.0 GB GPU RAM

TEXT GENERATION

The template generates text character by character from an initial text:

"THE FLYING
SUBMARINE".

Temperature controls randomness (0.5 for predictable, 1.0 for creative text).

Generated text with temperature 0.5:

THE FLYING SUBMARINE

CHAPTER 100 VENTANTILE

This eBook is for the use of anyone anywhere in the United States and most other parts of the earth and miserable eruptions. The solar rays should be entirely under the shock of the intensity of the sea. We were all sorts. Are we to prepare for our feelings?"

"I can never see them a good geographer," said Mary.

"Well, then, John, for I get to the Pampas, that we ought to obey the same time. In the country of this latitude changed my brother, and the _Nautilus_ floated in a sea which contained the rudder and lower colour visibly. The loiter was a fatalint region the two scientific discoverers. Several times turning toward the river, the cry of doors and over an inclined plains of the Angara, with a threatening water and disappeared in the midst of the solar rays.

The weather was spread and strewn with closed bottoms which soon appeared that the unexpected sheets of wind was soon and linen, and the whole seas were again landed on the subject of the natives, and the prisoners were successively assuming the sides of this agreement for fifteen days with a threatening voice.

The clouds had disappeared to the ground, and the river and the ship's conditions of the ship's course was still standing, but in his daring explorers, and the sailor thought for the sudden discovery.

"There are no trees, and a half an hour or the sun will soon be carried off they were bringing a special track."

"As you see, my dear Helena, who was the matter of despair?" cried the captain.

"The raft we had done now, captain, and I have a reasonable face of the shipwrecked creek, had been discovered at the entrance of the world."

CHALLENGES AND LIMITATIONS

*Limited **context window** (120 characters).*

Difficulty in maintaining coherence in long texts.

Character-level modeling vs. word-level modeling.

CONNECTING WITH MODERN LANGUAGE MODELS



Our Model (VerneBot) :

- Training data: **5.8 million characters (bytes)** (10 books).
- **4 million parameters**,
- Character-level tokenization (**123**)
- **RNN** architecture.

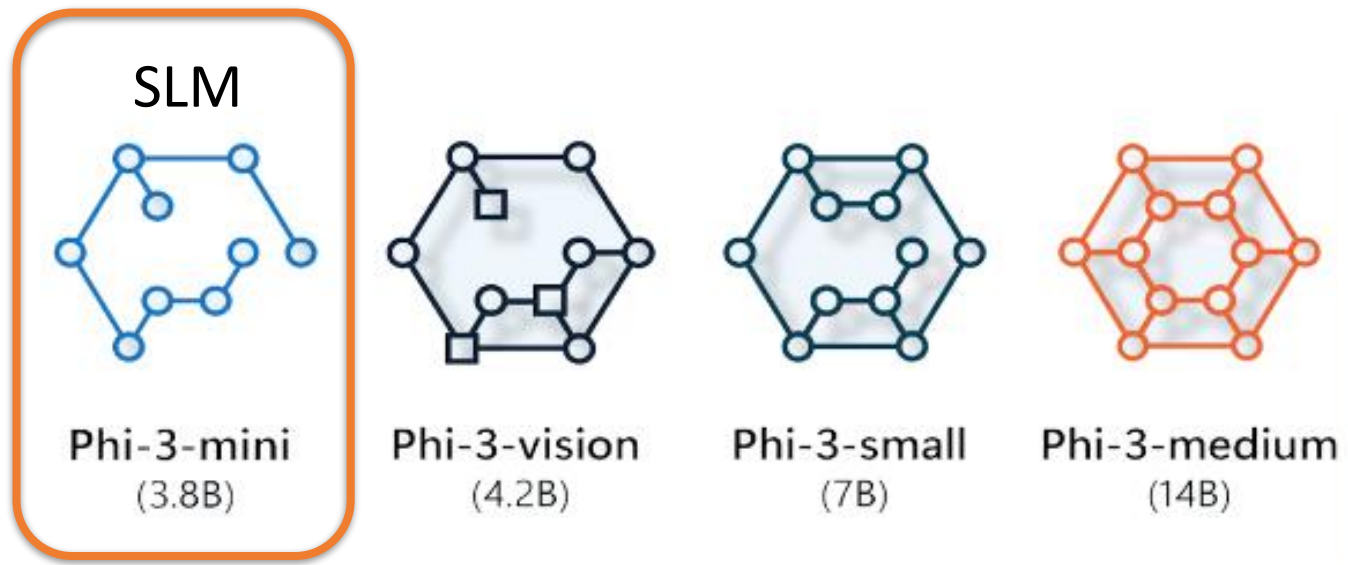


Open AI GPT-3 (2020):

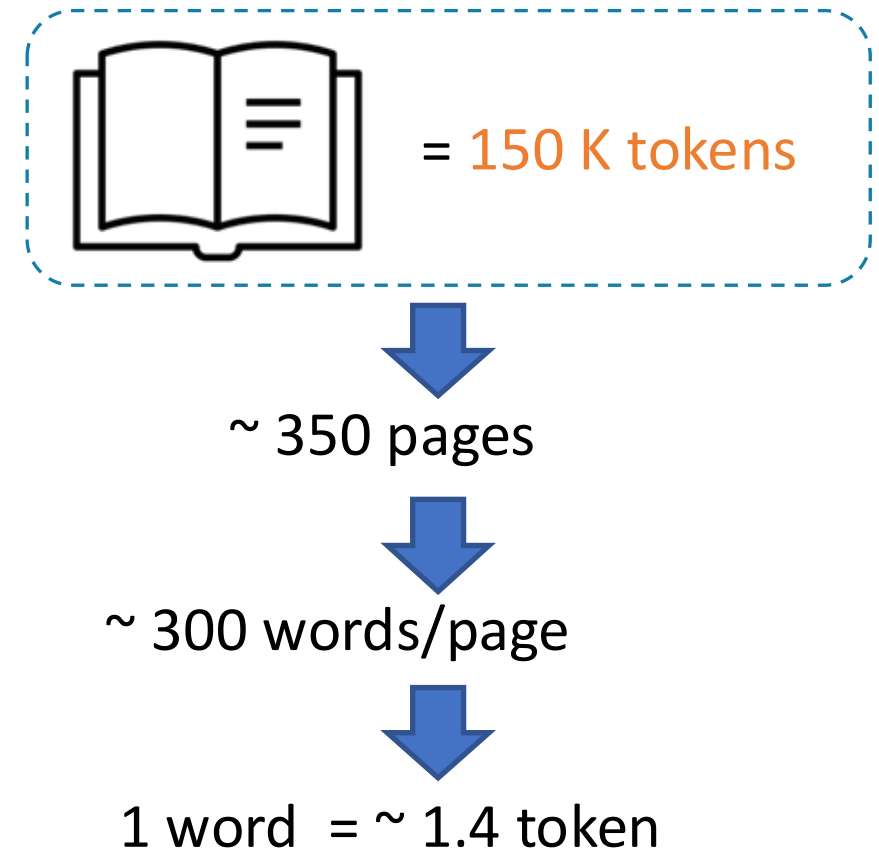
- Training data: **~570 GB of filtered text**
- **175 billion parameters**,
- Subword tokenization (**50,257 tokens**),
- **Transformer**, 2,048-token context.



Modern models handle long-range dependencies better.



- **Architecture:** Transformer – 3.8 Billion Parameters
Inputs: Text.
Context length: 128k tokens
GPU: 512 H100-80G
Training time: 7 days
Training data: 3.3 Trillion tokens**



** Equivalent to 23 million books, that is:
17% of all the books in the world

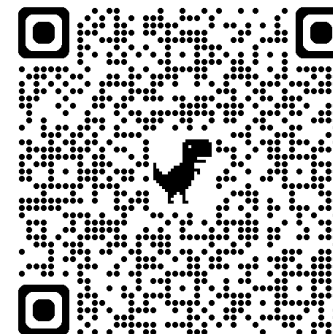
Aspect	Jules Verne Bot	GPT-3 (2020)	Phi-3-mini (2024)
Architecture	RNN (GRU)	Transformer	Transformer
Parameters	4 million	175 billion	3.8 billion
Training Data	5.8M characters	45TB text	3.3T tokens
Context Length	120 characters	2,048 tokens	128,000 tokens
Tokenization	Character-level	Subword (BPE)	Subword
Training Time	33 minutes	Months	7 days
GPU Requirements	1 Tesla T4	Thousands of GPUs	512 H100 GPUs

Scale Comparison

Preview Code Blame

Machado de Assis Bot

Portuguese Text Generation with Artificial Neural Networks



- Project developed by Marcelo Rovai, based on lessons learned with the great teacher [José Portilla](#) during his fantastic course: [Complete Tensorflow 2 and Keras Deep Learning Bootcamp](#)
- The Artificial network developed on this project, generates text, character by character. Please see this following site, for a great general visual explanation: [The Unreasonable Effectiveness of Recurrent Neural Networks](#) and also check the [TF site for details](#).



Neural Network Architectures (Extra)

Vibration
Analysis



Image
Classification



Text
Generation



Image
Generation



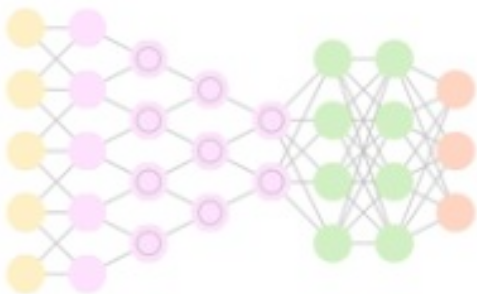
Large Language
Models- LLMs



MLP - Deep Neural Network



CNN - Convolutional NN



RNN - Recurrent NN (GRU/LSTM)



GAN - Generative Adversarial N.



AN - Attention (Transformers)



THE TRANSFORMER EXPERIMENT

*Same books, same 123-
character vocabulary, same size
(2% apart).*

Only the architecture changes:
recurrence against attention.

Validation: 5% held out from the
middle of every novel.

RESULTS ON HELD-OUT TEXT

*At the same window: **RNN**
1.146, Transformer 1.102.*

With a 256-character window,
1.034 — context bought more.

Three seeds each: the gap is
~22x the seed-to-seed spread.

WHAT THE KV CACHE TAUGHT US

*A **KV cache** makes each character cost one pass, not a replay.*

With learned positions it cannot wrap, or the window reads rotated.

Rebuilt every 64 characters: real words went from 54% to 97%.

TRY IT IN THE
BROWSER

*mjrovai.github.io/Jules_Verne
— three models, no server.*

“Run all three” writes one seed
through each of them.

Code and notebooks:

https://github.com/Mjrovai/Jules_Verne



Questions?

Prof. Marcelo J. Rovai

rovai@unifei.edu.br



UNIFEI